CS2403 Programming Languages

Subprograms

Adopt by apirak

Chung-Ta King
Department of Computer Science
National Tsing Hua University

(Slides are adopted from *Concepts of Programming Languages*, R.W. Sebesta)

國立清華大學
National Tsing Hua University

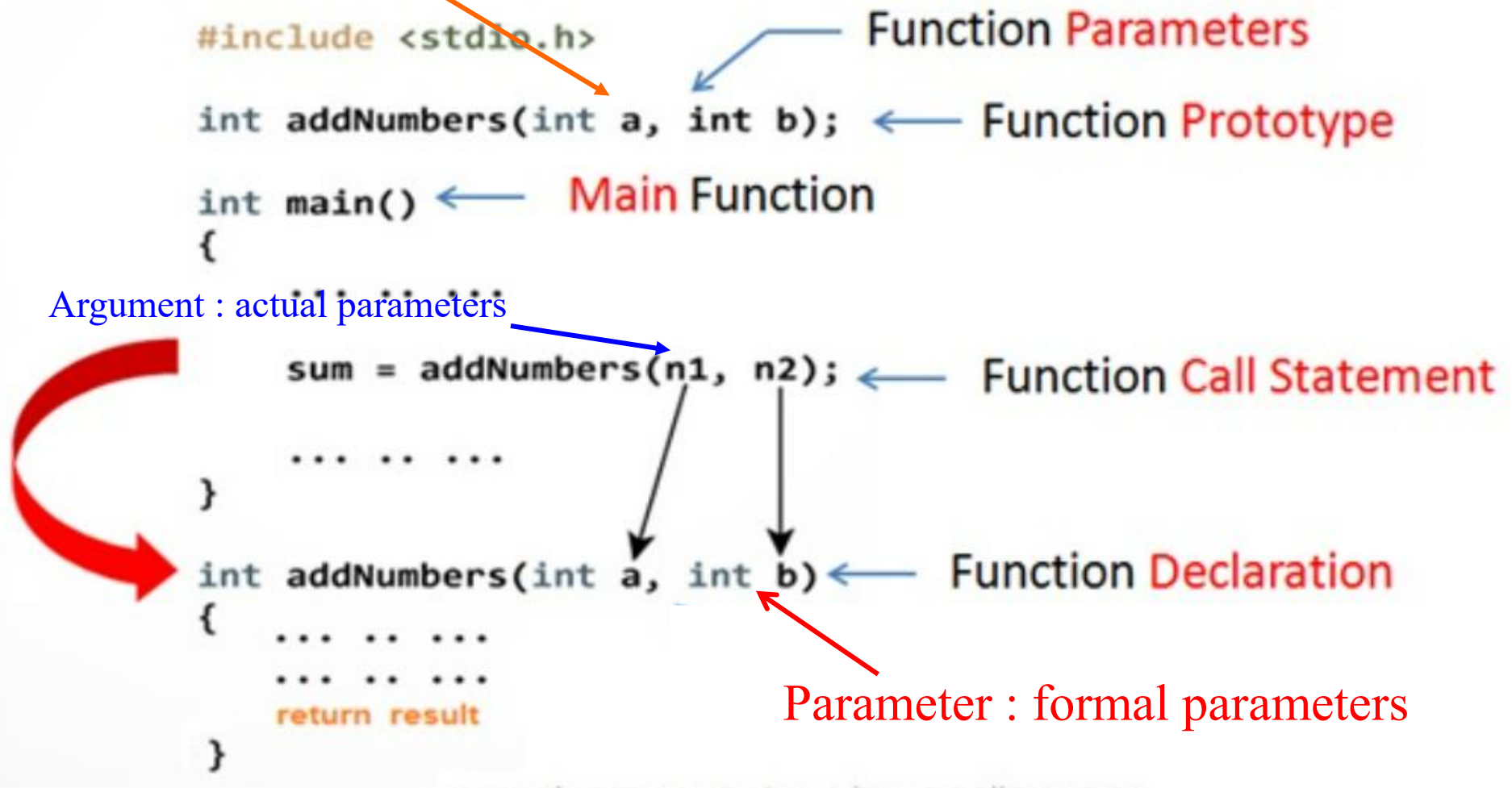
Overview

- Fundamentals of Subprograms (Sec. 9.2 - 9.4)
- Type of subprogram : procedure & function
- Parameter-Passing Methods (Sec. 9.5)
- Parameters That Are Subprograms (Sec. 9.6)
- Overloaded, Generic Subprograms (Sec. 9.7,9.8)
- Design Issues for Functions (Sec. 9.9)
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Fundamentals of Subprograms

Parameter : formal parameters

program organization on c/c++ :



Execution flows : exiting of function in "C/C++

terminating function with "}"

```
#include<iostream>

void greet() {
    // code
}

int main() {
    ... .. ...
    greet();
    ... .. ...
}
```

function call

terminating function with "return" instruction

```
#include<stdio.h>

int sum(int a, int b)
{
    return a+b;
     a = a-b;
}

int main()
{
    int ans;
    ans = sum(10,5);
    printf("10 + 5 = %d",ans);
    return 0;
}
```

Called function

Calling function

Execution will start from here after return

Overview

- Fundamentals of Subprograms (Sec. 9.2 - 9.4)
- Type of subprogram : procedure & function
- Parameter-Passing Methods (Sec. 9.5)
- Parameters That Are Subprograms (Sec. 9.6)
- Overloaded, Generic Subprograms (Sec. 9.7,9.8)
- Design Issues for Functions (Sec. 9.9)
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Two Categories of Subprograms/subroutine

- *Procedures*: collection of statements that define parameterized computations (no return value)

- *Functions:* structurally resemble procedures but are semantically modeled on math functions
  - Should be no side effects and return only one value to mimic mathematic functions
  - In practice, program functions have side effects
  - Functions define new user-defined operators

```
float power(float base, float exp)
result = 3.4 * power(10.0, x);
c.f. result = 3.4 * 10.0 ** x; (Perl)
```

Procedures : no return value subroutine

```cpp
#include <iostream>
// void means the function does not return a value to the caller
void printHi()
{
    std::cout << "Hi" << '\n';
    // This function does not return a value so no return statement is needed
}

int main()
{
    printHi(); // okay: function printHi() is called, no value is returned
    return 0;
}
```

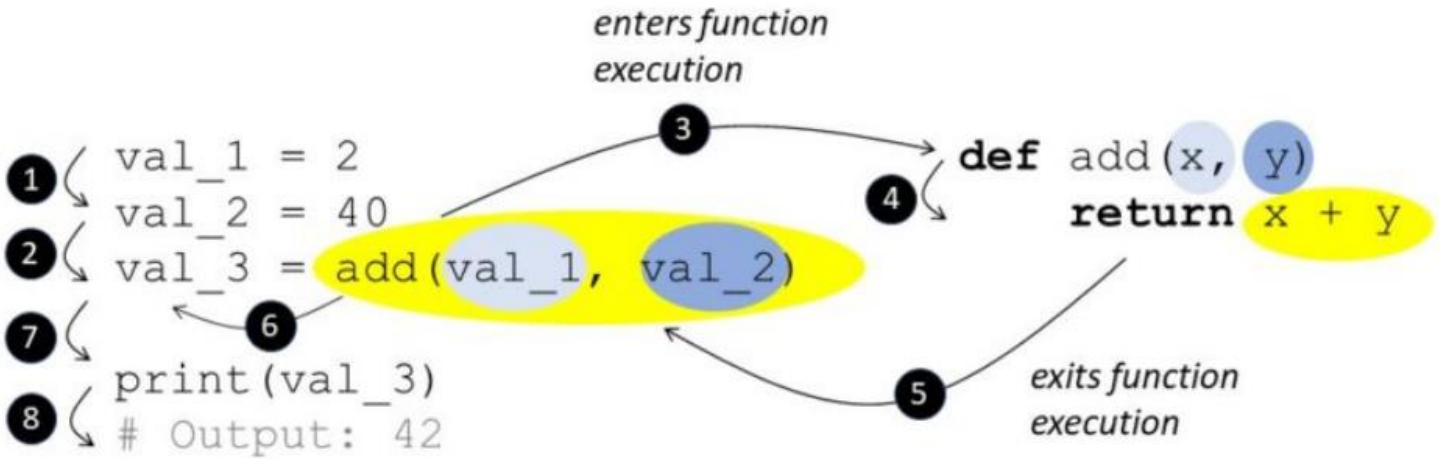
```python
#https://www.programiz.com/python-programming/online-compiler/
def f1():
  print ("Hello World - f1")
  return None

def f2():
  print ("Hello World -f2")
  return

def f3():
  print("Hello World -f3")

f1()
f2()
f3()
```

***Main Concept of Functions*:**

The routine copy its return results to the caller

return

finxter



- *Python **return** keyword → **exits function immediately** & returns value to caller.*
- *Optional return **value** or **expression**.*
- *No return value provided → returns None.*

Design Issue : Function (return value subroutine)

1. Return single value function

```
int f2( ) {

    return 0;
}
```

2. Return multiple value function

Function Return : C/C++

- Single return value
- Cannot return array

```
array1 :  0x7ffcf1f4f8e0  -> return array1
main array1() :  (nil)  -> (null)


array2 :  0x5651e5c242c0  -> return array2
main array2() :  0x7ffcf1f4fa50  -> return array2


array3 :  0x7ffcf1f4f960  -> return array3
main array3() :  0x7ffcf1f4f9e0  -> return array3
```

```c
#include <iostream>
#include <string.h>
char *array1()
{
    char *s = (char*)"return array1";
    char p[100];
    strcpy(p,s);
    printf("\narray1 :  %p  -> %s", p , p );
    return p;
}

char *array2()
{
    char *s = (char*)"return array2";
    char *p = (char*)malloc(100);
    strcpy(p,s);
    printf("\narray2 :  %p  -> %s", p , p );
    return p;
}

struct str {
    char p[100];
};
```

```c
struct str array3()
{
    char *s = (char*)"return array3";
    struct str x;
    strcpy( x.p,s);
    printf("\narray3 :  %p  -> %s", &x.p , x.p );
    return x;
}
int main()
{
    char *s1 = array1();
    printf("\nmain array1() :  %p  -> %s\n\n", s1 , s1 );

    char s2[100];
    s1 = array2();
    strcpy(s2, s1 );
    free(s1);
    printf("\nmain array2() :  %p  -> %s\n\n", s2 , s2 );

    struct str y;
    y = array3();
    printf("\nmain array3() :  %p  -> %s\n\n", &y.p , y.p );

    return 0;
}
```

return array c/c++ :

- return address(pointer)
- return structure copy value

國立清華大學
National Tsing Hua University

Return multi-value : Improve readability

main.py

```
1  #https://www.programiz.com/python-programming/online-compiler/
2  def return_multiple():
3      print("run return_multiple")
4      return 1, 2, 3
5  #
6  a, b, c = return_multiple()
7  print("a=" ,a , ",b = ",b , ",c = ", c)
8  #
9  x, _, z = return_multiple()
10 print("x=" ,x  , ",z = ", z)
11  #
12 return_multiple()
13
```

```
run return_multiple
a= 1 ,b =  2 ,c =  3
run return_multiple
x= 1 ,z =  3
run return_multiple
>
```

python

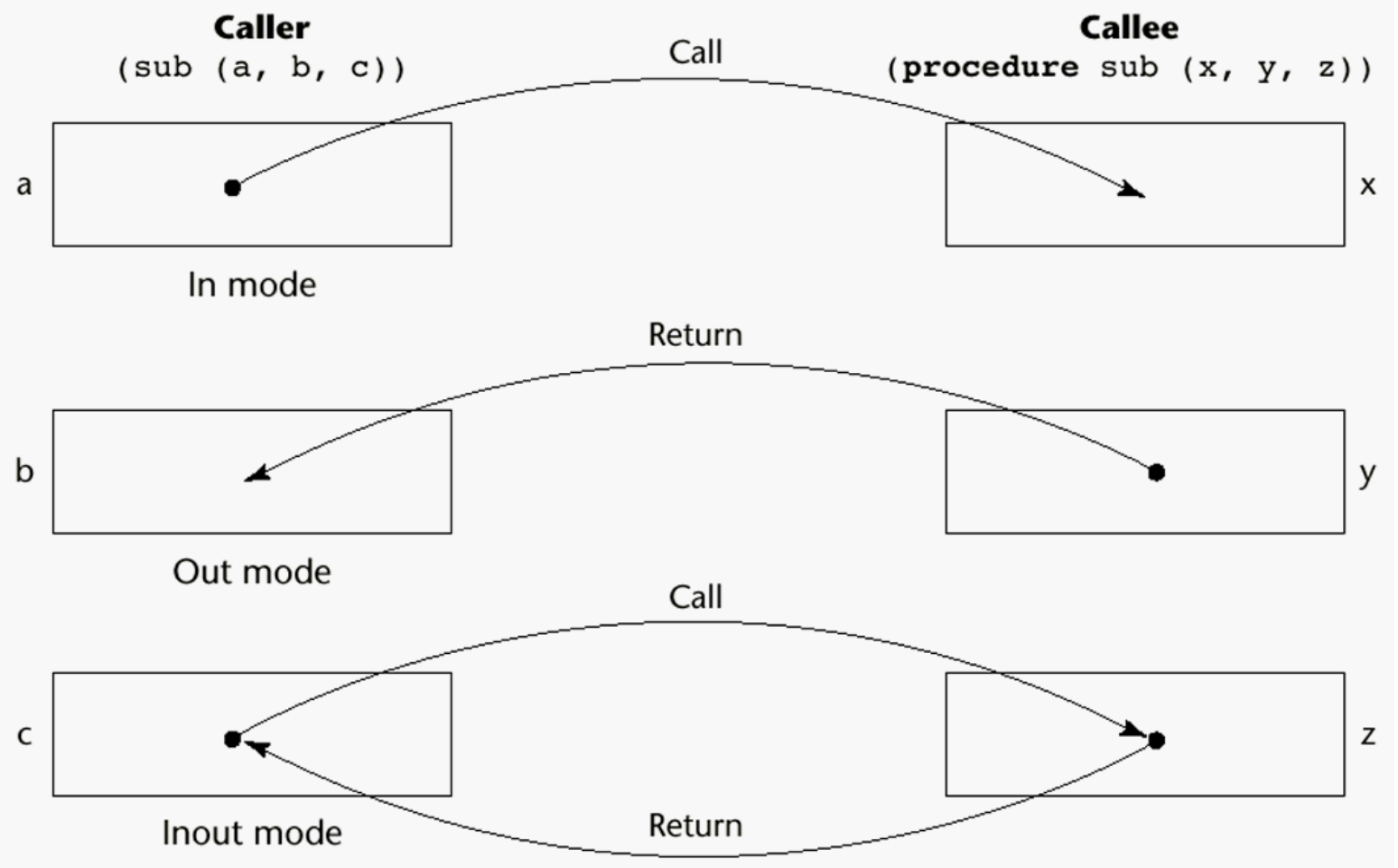
Overview

- Fundamentals of Subprograms (Sec. 9.2 - 9.4)
- Type of subprogram : procedure & function
- Parameter-Passing Methods (Sec. 9.5)
- Parameters That Are Subprograms (Sec. 9.6)
- Overloaded, Generic Subprograms (Sec. 9.7,9.8)
- Design Issues for Functions (Sec. 9.9)
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Most PL courses classify parameters Mode

- **in** – input only (pass by value)
- **out** – output only (pass by result)
- **In-out** – input + output (pass by ref, pass by value-result)

Parameter Passing Method
Semantic Models of Param Passing



Pass-by-Value (In Mode)

- The value of the actual parameter is used to initialize the corresponding formal parameter
  - Normally implemented by copying
- Disadvantages:
  - If by physical move: additional storage is required (stored twice) and the actual move can be costly (for large parameters)
  - If by access path: must write-protect in callee and accesses cost more (indirect addressing)

```c
// C program to illustrate call by value
#include <stdio.h>

void swapx(int x, int y)
{
    int t;
    t = x;
    x = y;
    y = t;
    printf("Inside Function:\nx = %d y = %d\n", x, y);
}
int main()
{
    int a = 10, b = 20;

    printf("before-In the Caller:\na = %d b = %d\n", a, b);
    // Pass by Values
    swapx(a, b);

    printf("after-In the Caller:\na = %d b = %d\n", a, b);

    return 0;
}
```

```
/tmp/fHcYpUEgqS.o
before-In the Caller:
a = 10 b = 20
Inside Function:
x = 20 y = 10
after-In the Caller:
a = 10 b = 20
```

Pass-by-Result (Out Mode)

- When a parameter is passed by result, no value is transmitted to the subprogram; the corresponding formal parameter acts as a local variable; its value is transmitted to caller's actual parameter when control is returned to the caller, by physical moving/copying
  - Require extra storage location and copy operation
  - C# -> mimic pass by result (however, it is pass by ref)
  - Ada -> implement pass by result

Comment Swap1()

```
2  //
3  using System;
4  class HelloWorld {
5   /*
6    static void Swap1(  out int x , out int y) {
7      int t =x; //problem no initial
8      x = y;
9      y = x;
10   }
11  */
12   static void Swap2( int x1,int y1, out int x2 , out int y2) {
13     y2 =x1;
14     x2 = y1;
15   }
16
17   static void Main() {
18    int x=20;
19    int y=30;
20    Console.WriteLine( "before x = {0} , y = {1}" , x,y);
21 //    Swap1(out x,out y);
22    Swap2(x,y,out x,out y);
23    Console.WriteLine( "after x = {0} , y = {1}" , x,y);
24   }
25 }
```

outmode： C#

```
main before -> g = 2
DoIt  before - > g =  2
DoIt  after - > g =  100
DoIt  afger - > p =  100
main after  -> g = 100
```

```
1  //
2  using System;
3  class HelloWorld {
4
5    static int g = 2;
6    static void DoIt(  out int p ) {
7        Console.WriteLine("DoIt  before - > g =  {0} ",g);
8      //  Console.WriteLine("DoIt  before - > p =  {0} ",p);
9        p = 100;
10       Console.WriteLine("DoIt  after - > g =  {0} ",g);
11       Console.WriteLine("DoIt  afger - > p =  {0} ",p);
12   }
13
14
15   static void Main() {
16     Console.WriteLine("main before -> g = {0}",g );
17     DoIt(out g );
18     Console.WriteLine("main after  -> g = {0}",g );
19
20   }
21 }
```

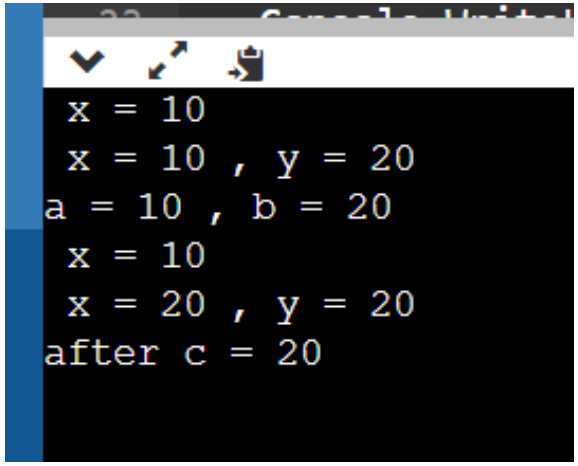
Internal technique c# , is not actual pass by value result

Pass-by-Result (Out Mode)

The same variable

- Potential problem: **`sub(p1, p1)`**
  - With the two corresponding formal parameters having different names, whichever formal parameter is copied back last will represent current value of p1

```
9  using System;
10 class HelloWorld {
11
12   static void sub(  out int x , out int y) {
13    x = 10;
14    Console.WriteLine( " x = {0}" , x);
15    y = 20;
16    Console.WriteLine( " x = {0} , y = {1}" , x,y);
17   }
18
19
20   static void Main() {
21    int a,b,c;
22    sub(out a,out b);
23    Console.WriteLine( "a = {0} , b = {1}" , a,b);
24    sub(out c,out c);
25    Console.WriteLine( "after c = {0}" , c);
26   }
27 }
28
```



```
1  -- https://www.tutorialspoint.com/compile_ada_online.php
2  with Ada.Text_IO; use Ada.Text_IO;
3  with Text_IO;
4  with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
5  procedure Hello is
6
7     A  : Integer := 1;
8     B  : Integer := 2;
9     C  : Integer := 3;
10
11 PROCEDURE getXY (x,y : out INTEGER) is
12 BEGIN
13     Put_Line("  before - > x = " & Integer'Image (x) );
14     Put_Line("  before - > y = " & Integer'Image (x) );
15     x := 20;
16     Put_Line("  getXY - > x = " & Integer'Image (x) );
17     y := 30;
18     Put_Line("  getXY - > y = " & Integer'Image (y) );
19 END getXY;
20
21 begin
22   Put_Line("before -> A = " & Integer'Image (A) );
23   Put_Line("before -> B = " & Integer'Image (B) );
24   getXY(A,B);
25   Put_Line("after -> A = " & Integer'Image (A) );
26   Put_Line("after -> B = " & Integer'Image (B) );
27
28 --   Put_Line("before -> C = " & Integer'Image (C) );
29 --   getXY(C,C);
30 --   Put_Line("after -> C = " & Integer'Image (C) );
31
32 end Hello;
33
```

Terminal

```
x86_64-linux-gnu-gcc-10 -c hello.adb
hello.adb:13:51: warning: "x" may be referenced before it
x86_64-linux-gnu-gnatbind-10 -x hello.ali
x86_64-linux-gnu-gnatlink-10 hello.ali -o hello
before -> A =  1
before -> B =  2
  before - > x =  0
  before - > y =  0
  getXY - > x =  20
  getXY - > y =  30
after -> A =  20
after -> B =  30
```

Uncomment getXY(C,C)

Ada checking

```ada
-- https://www.tutorialspoint.com/compile_ada_online.php
with Ada.Text_IO; use Ada.Text_IO;
with Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
procedure Hello is

   g  : Integer := 2;

PROCEDURE DoIt ( p :  out INTEGER  ) is
BEGIN
    Put_Line("DoIt  before - > g = " & Integer'Image (g) );
    Put_Line("DoIt  before - > p = " & Integer'Image (p) );
    p := 100;
    Put_Line("DoIt  after - > g = " & Integer'Image (g) );
    Put_Line("DoIt  after - > p = " & Integer'Image (p) );
END DoIt;

begin
  Put_Line("  main before - > g = " & Integer'Image (g) );

  DoIt( g );

  Put_Line("  main after  - > g = " & Integer'Image (g) );

end Hello;
```

```
x86_64-linux-gnu-gnatlink-10 hello.ali -o hello
main before - > g =  2
DoIt  before - > g =  2
DoIt  before - > p =  32552
DoIt  after - > g =  2
DoIt  after - > p =  100
  main after  - > g =  100
```

Ada

outmode : Ada vs C#

```
main before -> g = 2
DoIt  before - > g =  2
DoIt  after - > g =  100
DoIt  afger - > p =  100
main after  -> g = 100
```

C#

Inout Mode

- Pass by reference
- Pass by value result

Pass-by-Reference (Inout Mode)

- Pass an access path, also called *pass-by-sharing*
  - The called subprogram is allowed to access the actual parameter in the calling subprogram unit
- Advantage:
  - Passing process is efficient (no copying and no duplicated storage)
  - The main goal is to retrieve data from function
- Disadvantages
  - Slower accesses (compared to pass-by-value) to formal parameters due to indirect addressing
  - Potentials for unwanted changes to actual param.
  - Unwanted aliases (access to non-local)

Pass-by-Reference (Inout Mode)

- Aliases due to pass-by-reference:
  - Collisions between actual parameters, e.g., in C++

```
void fun(int &first, int &second)
fun(total, total);
```

  - Collisions between array and array elements

```
fun(list[i], list);
```

  - Collisions between formal parameters and nonlocal variables that are visible

```
int *global;
void main() { ... sub(global); ...}
void sub(int *param) {...}
```

`global` and `param` are aliases inside sub()

main.cpp

```
1  // C program to illustrate call by value
2  #include <stdio.h>
3
4  void swapx(int &x, int &y)
5  {
6      int t;
7      t = x;
8      x = y;
9      y = t;
10     printf("Inside Function:\nx = %d y = %d\n", x, y);
11 }
12 int main()
13 {
14     int a = 10, b = 20;
15
16     printf("before-In the Caller:\na = %d b = %d\n", a, b);
17     // Pass by Values
18     swapx(a, b);
19
20     printf("after-In the Caller:\na = %d b = %d\n", a, b);
21
22     return 0;
23 }
24
```

Array in c/c++
Pass by reference

main.cpp

```
1  // C program to illustrate call by value
2  #include <stdio.h>
3
4  int g=3;
5  void DoIt( int &x)
6  {
7      printf("\n before  :x = %d , g = %d", x, g);
8      g = 300;
9      printf("\n after  :x = %d , g = %d", x, g);
10 }
11 int main()
12 {
13     printf("\nbefore-In the Caller  -> g = %d", g);
14     // Pass by Values
15     DoIt( g );
16
17     printf("\nafter-In the Caller  -> g = %d", g);
18
19     return 0;
20 }
21
```

```
/tmp/fHcYpUEgqS.o
before-In the Caller  -> g = 3
 before  :x = 3 , g = 3
 after  :x = 300 , g = 300
after-In the Caller  -> g = 300
```

Pass-by-Value-Result (Inout Mode)

- A combination of pass-by-value and pass-by-result, sometimes called *pass-by-copy*
  - The value of the actual parameter is used to initialize the corresponding formal parameter, which then acts as a local variable
- Disadvantages:
  - Those of pass-by-result and pass-by-value

```
2  -- https://www.tutorialspoint.com/compile_ada_online.php
3  with Ada.Text_IO; use Ada.Text_IO;
4  with Text_IO;
5  with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
6  procedure Hello is
7
8     g  : Integer := 3;
9
10 PROCEDURE DoIt ( p : in out INTEGER  ) is
11 BEGIN
12     Put_Line("  DoIt-before - > g = " & Integer'Image (g) );
13     Put_Line("  DoIt-before - > p = " & Integer'Image (p) );
14     p := 40;
15     Put_Line("  DoIt-after - > g = " & Integer'Image (g) );
16     Put_Line("  DoIt-after - > p = " & Integer'Image (p) );
17 END DoIt;
18
19 begin
20   Put_Line("  main before - > g = " & Integer'Image (g) );
21
22   DoIt( g );
23
24   Put_Line("  main  before - > g = " & Integer'Image (g) );
25
26 end Hello;
27
```

```
main before - > g =  3
  DoIt-before - > g =  3
  DoIt-before - > p =  3
  DoIt-after - > g =  3
  DoIt-after - > p =  40
  main  before - > g =  40
```

Execute | Beautify | Share | Source Code | Help

```
3  with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
4  procedure Hello is
5   A : Integer ;
6   B  : Integer;
7   C : Integer;
8   result :  Integer;
9
10 FUNCTION myfunction ( p :  in out INTEGER  ) return Integer is
11 BEGIN
12     Put_Line("DoIt  before - > p = " & Integer'Image (p) );
13     p := p + 10;
14     C := 500;
15     Put_Line("DoIt  after - > p = " & Integer'Image (p) );
16     return p;
17 END myfunction;
18 begin
19   A := 20;
20   B := 5;
21   C := 10;
22   Put_Line("  main before - > A = " & Integer'Image (A) );
23   Put_Line("  main before - > B = " & Integer'Image (B) );
24    Put_Line("  main before - > C = " & Integer'Image (C) );
25   result := A * B + myfunction (C) + 20;
26   Put_Line("  main before - > result1 = " & Integer'Image (result) );
27
28   A := 20;
29   B := 5;
30   C := 10;
31   Put_Line("  main before - > A = " & Integer'Image (A) );
32   Put_Line("  main before - > B = " & Integer'Image (B) );
33   result := myfunction (C) + A*B+ 20;
```

Pass-by-Name

- The strangest feature of Algol 60, in retrospect, is the use of pass-by-name.
- The "define" stm in C/C++ mimics the "pass-by-name"

Let examin Jensen's Device problem

sum f_i for all i from 1 to n

where f_i **may be** $\mathbf{x}_i$ **,** $\mathbf{x}_i$ ***i** **,** $(\mathbf{x}_i + \mathbf{i} + 100)$

all i from 1 to 5 ,*where* f_i $(\mathbf{x}_i)$: *sum* = **x1+ x2 + x3 + x4 + x5**

all i from 1 to 5, *where* f_i ($\mathbf{x}_i$ *i): *sum* = **x1*1+ x2*2 + x3*3 + x4*4 + x5*5**

all i from 1 to 5 , f_i *be* $(\mathbf{x}_i + \mathbf{i} + 100)$: *sum* = **(x1+1+100)+ (x2+2+100) + (x3+3+100) + (x3+3+100) + (x3+3+100)**

Pass-by-Name : Let examines Jensen's Device problem

Sum(i, 1, n, x[i]*i)?

sum f_i for all i from 1 to n
where f_i be $x_i * i$, $(x_i + i + 100)$

```
real procedure Sum(j, lo, hi, Ej);
  value lo, hi;
  integer j, lo, hi; real Ej;
begin
  real S;
  S := 0;
  for j := lo step 1 until hi do
    S := S + Ej;
  Sum := S
end;
```

1. Using pass-by-reference or pass-by-value, we cannot do this because we would be passing in only a single value of **x[i]*i**
   sum(i, 0 , 5 , x[i]*i) ; sum(i, 0 , 5 , x[i]*i+100)
1. Using pass-by-name, the expression x[i]*i is passed in without evaluation

Macro #Define in c/c++

```c
#define AREA_OF_CIRCLE(r) (3.141*r*r)
main()
{
   float ra = 5.7;
   printf("The Area of circle with radius %f is %f\n", r, AREA_OF_CIRCLE(ra));
}
```

replaced by

```c
printf("The Area of circle with radius %f is %f\n", r, (3.141*ra*ra))
```

```c
#define HORLINE for( i = 0 ; i < 79 ; i++ ) \
                    printf ( "%c", 196 ) ;
main( )
{
   int i;
   clrscr();
   HORLINE
}
```

replaced by

```c
for( i = 0 ; i < 79 ; i++ )
        printf ( "%c", 196 ) ;
```

```c
//
#include <stdio.h>

#define  Sum(j, lo, hi, Ej,S) {  S = 0; \
                                  for( j = lo ;  j < hi ; j++ ) { \
                                    S = S + Ej; \
                                    printf("\nEj= %d , j= %d",Ej,j ); \
                                  } \
                                }
int main()
{
    int outx;
    int i;
    int x[] = { 1,2,3,4,5 };
    Sum( i , 0 ,5 , x[i], outx );
    printf( "\n outx = %d", outx );

    int outz;
    Sum( i , 0 ,5 , x[i]*i, outz );
    printf( "\n outz = %d", outz );

    int outy;
    int k;
    int y[] = { 11,12,13,14,15 };
    Sum( k , 0 ,5 , (y[k]+k+100), outy );
    printf( "\n outy = %d", outy );
    return 0;
}
```

```
/tmp/fHcYpUEgqS.o
Ej= 1 , j= 0
Ej= 2 , j= 1
Ej= 3 , j= 2
Ej= 4 , j= 3
Ej= 5 , j= 4
outx = 15
Ej= 0 , j= 0
Ej= 2 , j= 1
Ej= 6 , j= 2
Ej= 12 , j= 3
Ej= 20 , j= 4
 outz = 40
Ej= 111 , j= 0
Ej= 113 , j= 1
Ej= 115 , j= 2
Ej= 117 , j= 3
Ej= 119 , j= 4
 outy = 575
```

problem

Pass-by-name

```
array A [1..100] of int;
int i=5;
foo(A[i],i);
print A[i]; # prints A[6]
...          # so prints 7

# good example
proc foo (name B,name k) {
   k = 6;
   B = 7;
}

# text substitution does this
proc foo {
   i = 6;
   A[i] = 7;
}
```

```
array A [1..100] of int;
int i=5;
foo(A[i]);
print A[i]; # prints A[5]
...          # not sure what

# a problem here...
proc foo (name B) {
   int i = 2;
   B = 7;
}

proc foo {
   int i = 2;
   A[i] = 7;
}
```

Pass-By-Name Security Problem (Severe)

1. A sample program:

```
procedure swap (a, b);
integer a, b, temp;
begin
  temp := a;
  a := b;
  b:= temp
end;
```

Function source code

2. Effect of the call `swap(x, y)`:

```
temp := x;
x := y;
y := temp
```

3. Effect of the call `swap(i, x[i])`:

```
temp := i;
i := x[i];
x[i] := temp
```

It doesn't work! For example:

Multidimensional Arrays as Param

- If a multidimensional array is passed to a subprogram and the subprogram is separately compiled, compiler needs to know declared size of that array to build storage mapping function
  - A storage-mapping function for row-major matrices:
    address(mat[i,j])=
    address(mat[0,0]) + i * #_columns + j
    - Only needs to know #_columns

Multidimensional Arrays as Param

- For C and C++:
  - Programmer is required to include the declared sizes of all but the first subscript in the actual parameter

```
void fun(int mat[][10]) { ... }
void main() {
  int mat[5][10];    ...
  fun(mat);    ... }
```

  - Disallows writing flexible subprograms
  - Solution: pass a pointer to the array and sizes of the dimensions as other parameters; user must include storage mapping function in terms of size parameters

Multidimensional Arrays as Param

- Java and C#
  - Arrays are objects; they are all single-dimensioned, but the elements can be arrays
  - Each array inherits a named constant (**`length`** in Java, **`Length`** in C#) that is set to the length of the array when the array object is created, e.g., in Java

```
float sumer(float mat[][]) {
  for(int row=0; row<mat.length; row++)
    for(int col=0; col<mat[row].length;
                col++)
      sum += mat[row][col];
```

```
using System;
class system
{
    static void Main(string[] args)
    {
            int[,] ProcArea = new int[5, 5];
            RandomFill( ProcArea, 5, 5);
             // display new matrix in 5x5 form
            int i, j;
            for (i = 0; i < 5; i++)
            {
                for (j = 0; j < 5; j++)
                    Console.Write("{0}\t", ProcArea[i, j]);
                Console.WriteLine();
            }

    }

    public static void RandomFill( int[,] array, int rows, int columns)
    {
        Random rand = new Random();
        //Fill randomly
        for (int r = 0; r < rows; r++)
        {
            for (int c = 0; c < columns; c++)
            {
                if (rand.NextDouble() < 0.55)
                {
                    array[r, c] = 1;
                }
                else
                {
                    array[r, c] = 0;
                }
            }
        }
    }
}
```

Design Issue for Functions

- Parameter Passing Method (Sec. 9.5)
  - Pass by value, by reference , by name , by Value-Result , by result
- Parameters That Are Subprograms (Sec. 9.6)
- Parameter Passing Binding (parameter order) & Return values (Categories of Subprograms) (Sec. 9.2 – 9.4)
- Return values (Categories of Subprograms)
- Nested subroutine
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Subprogram Names as Parameters

- It is sometimes convenient to pass subprogram names as parameters
  → pass a computation to a subprogram
- Are parameter types checked?
  - C and C++: functions cannot be passed as parameters but pointers to functions can be passed and types of function pointers include the types of the parameters, so parameters can be type checked
  - Java does not allow method names to be passed as parameters

```c
#include <stdio.h>
void add(int a, int b)
{
    printf("Addition is %d\n", a+b);
}
void subtract(int a, int b)
{
    printf("Subtraction is %d\n", a-b);
}
void multiply(int a, int b)
{
    printf("Multiplication is %d\n", a*b);
}
void dofun( void (*fun_ptr_arr)(int, int) , int a,int b )
{
    printf("\npredo something on dofun()\n");
    (*fun_ptr_arr)(a, b);
    printf("\npostdo something on dofun()");
}
int main()
{
    // fun_ptr_arr is an array of function pointers
    void (*fun_ptr_arr[])(int, int) = {add, subtract, multiply};
    unsigned int ch, a = 15, b = 10;
    printf("Enter Choice: 0 for add, 1 for subtract and 2 "
           "for multiply (a=%d , b=%d) \n",a,b);
    scanf("%d", &ch);
    dofun(fun_ptr_arr[ch] , a,b );
    return 0;
}
```

```
/tmp/fHcYpUEgqS.o
Enter Choice: 0 for add, 1 for subtract and 2 for multiply (a=15 , b=10)
1
predo something on dofun()
Subtraction is 5

postdo something on dofun()
```

Function pointer in c/c++

Subprogram Names as Parameters

```
1  //https://www.programiz.com/javascript/online-compiler/
2  function sayHello(param) {
3      console.log("hello", param);
4      param();
5      return "Hiii Geeks for Geeks"
6  }
7
8  // Function address
9  function smaller() {
10     console.log("Is everything alright")
11 }
12
13 // Function call
14 const returnHello = sayHello(smaller)
15 console.log(returnHello)
16
```

```
node /tmp/ri4BPQheGm.js
hello [Function: smaller]
Is everything alright
Hiii Geeks for Geeks
```

Java script

```
using System;
using System.Collections.Generic;
class Program
{
    // define a method that returns sum of two int numbers
    static int calculateSum(int x, int y)
    {
        return x + y;
    }

    // define a delegate
    public delegate int myDelegate(int num1, int num2);

    static void f1( myDelegate d ,int a,int b )
    {
        Console.WriteLine("pre-process f1");
        int result = d(a, b);
        Console.WriteLine(result);
        Console.WriteLine("post-process f1");
    }

    static void f2( myDelegate d ,int a,int b=200 )
    {
        Console.WriteLine("pre-process f2");
        int result = d(a, b);
        Console.WriteLine(result);
        Console.WriteLine("post-process f2");
    }

    static void Main()
    {
        f1(calculateSum,10,20);
        f2(calculateSum,100);
    }
}
```

```
pre-process f1
30
post-process f1
pre-process f2
300
post-process f2
```

C# delegate function

Java script

Design Issue for Functions

- Parameter Passing Method (Sec. 9.5)
  - Pass by value, by reference , by name , by Value-Result , by result
- Parameters That Are Subprograms (Sec. 9.6)
- Parameter Passing Binding (parameter order) (Sec. 9.2 – 9.4)
- Return values (Categories of Subprograms) (Sec. 9.2 – 9.4)
- Overloaded, Generic Subprograms(Sec. 9.7,9.8)
- Nested subroutine
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Basic Definitions

- A *subprogram definition* describes the interface to and actions of the subprogram
  - A *subprogram header* is the 1st part of the definition, including name, subprogram kind, formal parameters

    `void adder(parameters)`
  - Number, order, types of formal parameters are called *parameter profile* (*signature*) of the subprogram
- A *subprogram declaration* provides parameter profile and return type, but not the body
  - *Prototypes* in C and C++; for compilers to see
- A *subprogram call* jumps to the subprogram

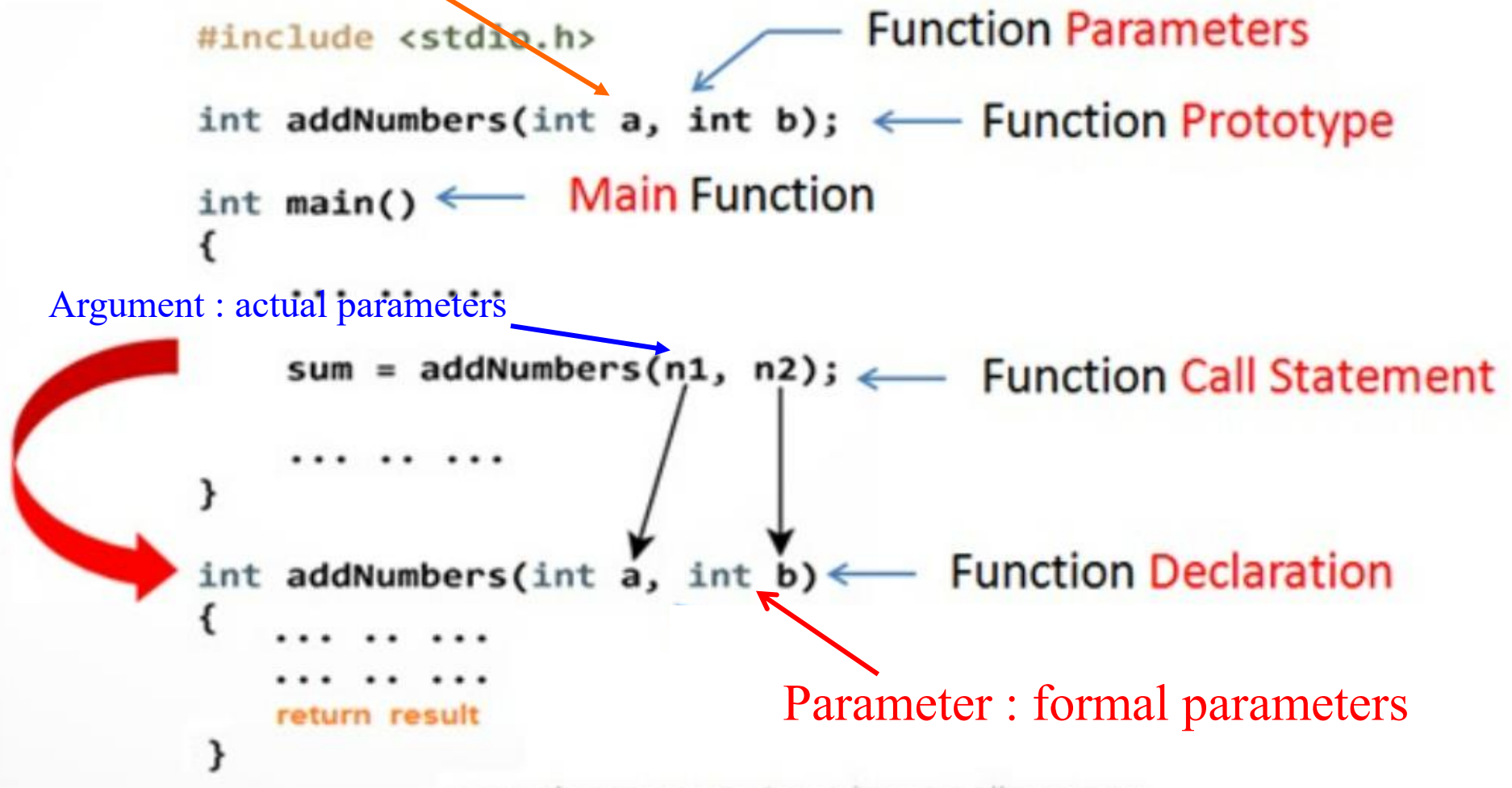
Parameters Types

- A *formal parameter* is a dummy variable listed in subprogram header and used in the subprogram
  - Usually bound to storage when subprogram is called
- An *actual parameter* represents a value or address used in the subprogram call statement

Fundamentals of Subprograms

Parameter : formal parameters

program organization on c/c++ :



Binding Actual to Formal Parameter

- Positional parameters
  - Binding by position: the first actual parameter is bound to the first formal parameter and so forth
- Keyword parameters: for long parameter list
  - Name of formal parameter to which an actual parameter is to be bound is specified

```
sumer(my_length, list=my_array,
      sum = my_sum)
```

-- Python

  - Pros: Parameters can appear in any order
  - Cons: User must know the formal parameter's names

Positional parameters c/c++ vs swift

```cpp
#include <iostream>
using namespace std ;
// function without prototype
void addition ( int a , int b ,const char *c ) {
    cout << a <<endl;
    cout << b <<endl;
    cout << c <<endl;
}
int main ( ) {
    // calling the function before declaration.
    addition ( 10 , 20,"c (a:10,b:20)" ) ;
    addition ( 20 , 10,"c (a:10,b:20)" ) ;
   // addition ( "c (a:10,b:20)", 20 , 10 ) ;
    return 0 ;
}
// defining function
```

main.swift

```swift
////https://www.onlinegdb.com/online_swift_compiler
func addition(a: Int, b: Int, c:String) -> Int {
    print( a)
    print( b )
    print (c )
    return a + b
}

addition( a:10, b:20,c:"order  (a:10, b:20, c)")
//addition( 10, 20,"order  (a:10, b:20, c)")
//addition( b:20,c:"order  (a:10, b:20, c)",a:10)

```

https://www.onlinegdb.com/online_swift_compiler

Keyword parameters: for long list parameter and having the same the

main.py

```
1  #https://www.programiz.com/python-programming/online-compiler/
2  def my_function( a,b,c):
3    print("a = : %2d " % (a))
4    print("b = : %2d " % (b))
5    print("c = : %s " % (c))
6    print("")
7
8  my_function( a=10,b=20,c="a=10,b=20 ")
9  my_function( b=20,a=10,c="b=20,a=10 ")
10 my_function( c="b=20,a=10 ",b=20,a=10)
11
```

```
a = : 10
b = : 20
c = : a=10,b=20

a = : 10
b = : 20
c = : b=20,a=10

a = : 10
b = : 20
c = : b=20,a=10
```

Unlimited parameter

c/c++

main.cpp

```c
#include <stdarg.h>
#include <stdio.h>

double average(int count, ...) {
    va_list ap;
    int j;
    double sum = 0;

    va_start(ap, count); /* Before C23: Requires the
    for (j = 0; j < count; j++) {
        sum += va_arg(ap, double); /* Increments ap t
    }
    va_end(ap);

    return sum / count;
}

int main(int argc, char const *argv[]) {
    printf("%f\n", average(3, 1.5, 2.3, 3.1));
    return 0;
}
```

Unlimited parameter

c/c++

```cpp
#include <iostream>
#include <cstdarg>

void simple_printf(const char* fmt...)      // C-style "const char* fmt, ..."
{
    va_list args;
    /* Before C23: Requires the last fixed parameter (to get the address) */
    va_start(args, fmt);

    while (*fmt != '\0') {
        if (*fmt == 'd') {
            int i = va_arg(args, int);
            std::cout << i << '\n';
        } else if (*fmt == 'c') {
            // note automatic conversion to integral type
            int c = va_arg(args, int);
            std::cout << static_cast<char>(c) << '\n';
        } else if (*fmt == 'f') {
            double d = va_arg(args, double);
            std::cout << d << '\n';
        }
        ++fmt;
    }

    va_end(args);
}

int main()
{
    simple_printf("dcff", 3, 'a', 1.999, 42.5);
}
```

Unlimited parameter JAVA

```java

//https://www.programiz.com/java-programming/online-compiler/
class HelloWorld {
    // Variadic methods store any additional arguments they rece
    // Consequentially, `printArgs` is actually a method with on
    // variable-length array of `String`s.
    private static void printArgs(String... strings) {
        for (String string : strings) {
            System.out.println(string);
        }
    }

    public static void main(String[] args) {
        printArgs("hello");          // short for printArgs(["he
        printArgs("hello", "world","3.0"); // short for printArg
    }
}
```

https://www.programiz.com/java-programming/online-compiler/

Unlimited parameter C#

```
using System;

class Program
{
    static int Foo(int a, int b, params int[] args)
    {
        // Return the sum of the integers in args, ignoring a and b.
        int sum = 0;
        foreach (int i in args)
            sum += i;
        return sum;
    }

    static void Main(string[] args)
    {
        Console.WriteLine(Foo(1, 2));  // 0
        Console.WriteLine(Foo(1, 2, 3, 10, 20));  // 33
        int[] manyValues = new int[] { 3, 10, 20 };
        Console.WriteLine(Foo(1, 2, manyValues));  // 42
    }
}

```

Formal Parameter Default Values

- In certain languages (e.g., C++, Python, Ruby, Ada, PHP), formal parameters can have default values (if no actual parameter is passed)
  - In C++, default parameters must appear last because parameters are positionally associated

```
float comput_pay (float income,
    float tax_rate, int exemptions = 1)
pay = comput_pay (20000.0, 0.15);
                          -- exemptions takes 1
```

Default parameter in c/c++

```cpp
#include <iostream>
using namespace std;

int sum(int x, int y, int z = 0, int w = 0) //
{
    return (x + y + z + w);
}

// Driver Code
int main()
{
    // Statement 1
    cout << sum(10, 15) << endl;

    // Statement 2
    cout << sum(10, 15, 25) << endl;

    // Statement 3
    cout << sum(10, 15, 25, 30) << endl;
    return 0;
}
```

What the meaning of default parameter ?

Java not support but it support default parameter function

Basic Definitions

- Prototype
- Caller order
- declare in class template

Prototype in c/c++

```
# include < iostream >
using namespace std ;
// function prototype
void divide ( int , int ) ;
int main ( ) {
   // calling the function before declaration.
   divide ( 10 , 2 ) ;
   return 0 ;
}
// defining function
void divide ( int a , int b ) {
   cout < < ( a / b ) ;
}
```

main.cpp

```
1  #include <iostream>
2  using namespace std ;
3  // function without prototype
4  void divide ( int a , int b ) {
5      cout << ( a / b ) ;
6  }
7  int main ( ) {
8      // calling the function before declaration.
9      divide ( 10 , 2 ) ;
10     return 0 ;
11 }
12 // defining function
13
```

Without Prototype lang.

main.cpp

```cpp
#include <iostream>
using namespace std ;
// function without prototype
void divide ( int a , int b ) {
    cout << ( a / b ) ;
}
int main ( ) {
    // calling the function before d
    divide ( 10 , 2 ) ;
    return 0 ;
}
// defining functi
```

c/c++

```pascal
program exFunction;
var
   a, b, ret : integer;
(*function definition *)
function max(num1, num2: integer): integer;
var
   (* local variable declaration *)
   result: integer;
begin
   if (num1 > num2) then
      result := num1
   else
      result := num2;
   max := result;
end;

begin
   a := 100;
   b := 200;
   (* calling a function to get max value *)
   ret := max(a, b);
   writeln( 'Max value is : ', ret );
end.
```

pascal

```python
def greet():
    print('Hello World!')

# call the function
greet()

print('Outside function')
```

python

Prototype : modern lang. c/c++ , java, c# declare in class template

Outside Example

```cpp
class MyClass {        // The class
  public:              // Access specifier
    void myMethod();   // Method/function declaration
};

// Method/function definition outside the class
void MyClass::myMethod() {
  cout << "Hello World!";
}

int main() {
  MyClass myObj;     // Create an object of MyClass
  myObj.myMethod();  // Call the method
  return 0;
}
```

```java
class Main {

  // method with no parameter
  public void display1() {
    System.out.println("Method without parameter");
  }

  // method with single parameter
  public void display2(int a) {
    System.out.println("Method with a single parameter: " + a);
  }

  public static void main(String[] args) {

    // create an object of Main
    Main obj = new Main();

    // calling method with no parameter
    obj.display1();

    // calling method with the single parameter
    obj.display2(24);
  }
}
```

Design Issue for Functions

- Parameter Passing Method (Sec. 9.5)
  - Pass by value, by reference , by name , by Value-Result , by result
- Parameters That Are Subprograms (Sec. 9.6)
- Parameter Passing Binding (parameter order) (Sec. 9.2 – 9.4)
- Return values (Categories of Subprograms) (Sec. 9.2 – 9.4)
- Overloaded, Generic Subprograms(Sec. 9.7,9.8)
- Nested subroutine
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Overloaded Subprograms

- One that has the same name as another subprogram in same referencing environment
  - Every version of an overloaded subprogram has a unique protocol; meaning decided by actual param

```cpp
#include <iostream>
using namespace std;

void show(int i)
{
    cout << i << endl;
}

void show(const char *s)
{
    cout << s << endl;
}

void show(double f, const char *s=NULL)
{
    cout << f << endl;
    if(s!=NULL)
        cout << s << endl;
}

// Driver Code
int main()
{
    // Statement 1
    show(10+20);
    show("msg:10+20");
    show(30.4);
    show(30.5,"data from avg value");

    return 0;
}
```

```
30
msg:10+20
30.4
30.5
data from avg value
```

Overloading function with default parameters

```cpp
#include <iostream>
using namespace std;

int sum(int x, int y, int z = 0, int w = 0) //
{
    return (x + y + z + w);
}

// Driver Code
int main()
{
    // Statement 1
    cout << sum(10, 15) << endl;

    // Statement 2
    cout << sum(10, 15, 25) << endl;

    // Statement 3
    cout << sum(10, 15, 25, 30) << endl;
    return 0;
}
```

Homework convert to java and compare to c++ (read/writability)

Overloaded Subprograms

- Problem with default parameters:

```
void fun(float b = 0.0);
Void fun();
```

```
#include <iostream>
using namespace std;


void shown(int x ) //assigning default values t
{
    cout << "x = "<< x << endl;
}

void shown(int x, int y) //assigning default va
{
    cout << "x = "<< x << endl;
    cout << "y = "<< y << endl;
}
/*
void shown(int x, int y,float f=50.0) //assigni
{
    cout << "x = "<< x << endl;
    cout << "y = "<< y << endl;
    cout << "f = "<< f << endl;}
 }
 */
// Driver Code
int main()
{
    shown(10);
    shown(100,200);
   // shown(100,200,1.5);

    return 0;
}
```

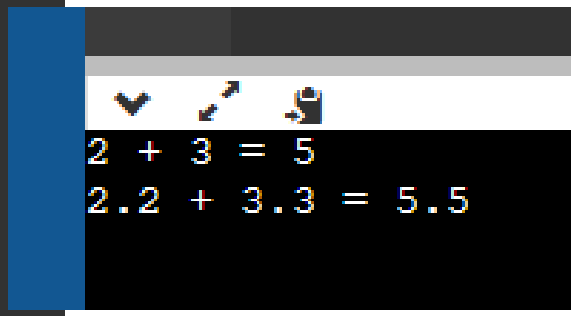
Generic Subprograms (Template function in C/C++

- Software reuse → create one subprogram that works on different types of data, e.g., sorting on arrays of different element types
- A *polymorphic* subprogram takes parameters of different types on different activations
  - Overloaded subprograms provide *ad hoc polymorphism*
  - *Parametric polymorphism:* a subprogram that takes a generic (type-less) parameter that is used in a type expression that describes the type of the parameters

Generic Subprograms (Template function in C/C++)

main.cpp

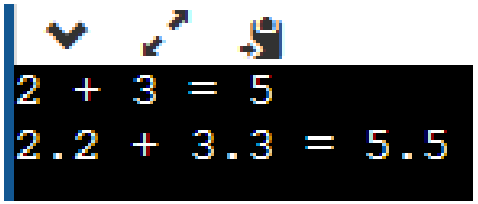
```cpp
#include <iostream>
using namespace std;


int add(int num1, int num2) {
    int v = (num1 + num2);
    return v;
}

double add(double num1, double num2) {
    double v = (num1 + num2);
    return v;
}

int main() {
    int result1;
    double result2;
    // calling with int parameters
    result1 = add(2, 3);
    cout << "2 + 3 = " << result1 << endl;
    // calling with double parameters
    result2 = add(2.2, 3.3);
    cout << "2.2 + 3.3 = " << result2 << endl;

    return 0;
}
```



Generic Subprograms (Template function in C/C++)

main.cpp

```cpp
#include <iostream>
using namespace std;

template <typename T>
    T add(T num1, T num2) {
    T v = (num1 + num2);
    return v;
}

int main() {
    int result1;
    double result2;
    // calling with int parameters
    result1 = add<int>(2, 3);
    cout << "2 + 3 = " << result1 << endl;
    // calling with double parameters
    result2 = add<double>(2.2, 3.3);
    cout << "2.2 + 3.3 = " << result2 << endl;

    return 0;
}
```



```
#include<iostream>

template<typename T>
T add(T num1, T num2) {
  return (num1 + num2);
}

int main() {
  ... ... ...

  result1 = add<int>(2,3);
  ... ... ...

  result2 = add<double>(2.2,3.3);
  ... ... ...

}
```

```
int add(int num1, int num2) {
    return (num1 + num2);
}
```

```
double add(double num1, double num2) {
    return (num1 + num2);
}
```

Design Issue for Functions

- Parameter Passing Method (Sec. 9.5)
  - Pass by value, by reference , by name , by Value-Result , by result
- Parameters That Are Subprograms (Sec. 9.6)
- Parameter Passing Binding (parameter order) (Sec. 9.2 – 9.4)
- Return values (Categories of Subprograms) (Sec. 9.2 – 9.4)
- Overloaded, Generic Subprograms(Sec. 9.7,9.8)
- Nested subroutine
- User-Defined Overloaded Operators (Sec. 7.3 &9.10)
- Coroutines (Sec. 9.11)

Overloaded Operators

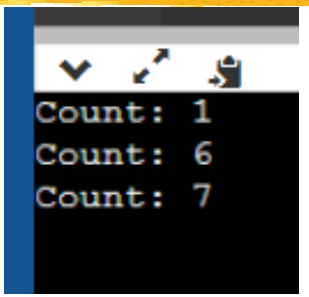
- Use of an operator for more than one purpose is called *operator overloading*
- Some are common (e.g., + for `int` and `float`)
- Some are troublesome (e.g., `*` in C and C++)
  - Loss of compiler error detection (omission of an operand should be a detectable error)
  - Some loss of readability

Overloaded Operators (cont.)

- C++/C# allow user-defined overloaded operator
  - Users can define nonsense operations
  - Readability may suffer, even when operators make sense, e.g., need to check operand types to know

+	-	*	/	%	^	&	|	~
!	=	<	>	+=	-=	*=	/=	%=
^=	&=	|=	<<	>>	<<=	>>=	==	!=
<=	>=	&&	||	++	--	,	->*	->
()	[]	new	delete	new[]	delete[]			

```cpp
#include <iostream>
using namespace std;
class Count {
   private:
    int value;
   public:
    Count(int a=0) : value(a) {}
    void operator ++ () {
        ++value;
    }
    void display() {
        cout << "Count: " << value << endl;
    }
    Count operator+(Count const& obj)
    {
        Count res;
        res.value = value + obj.value;
        return res;
    }
};

int main() {
    Count count1,count2(5),count3;
    ++count1;
    count1.display();
    ++count2;
    count2.display();
    count3 = count1 + count2;
    count3.display();
    return 0;
}
```

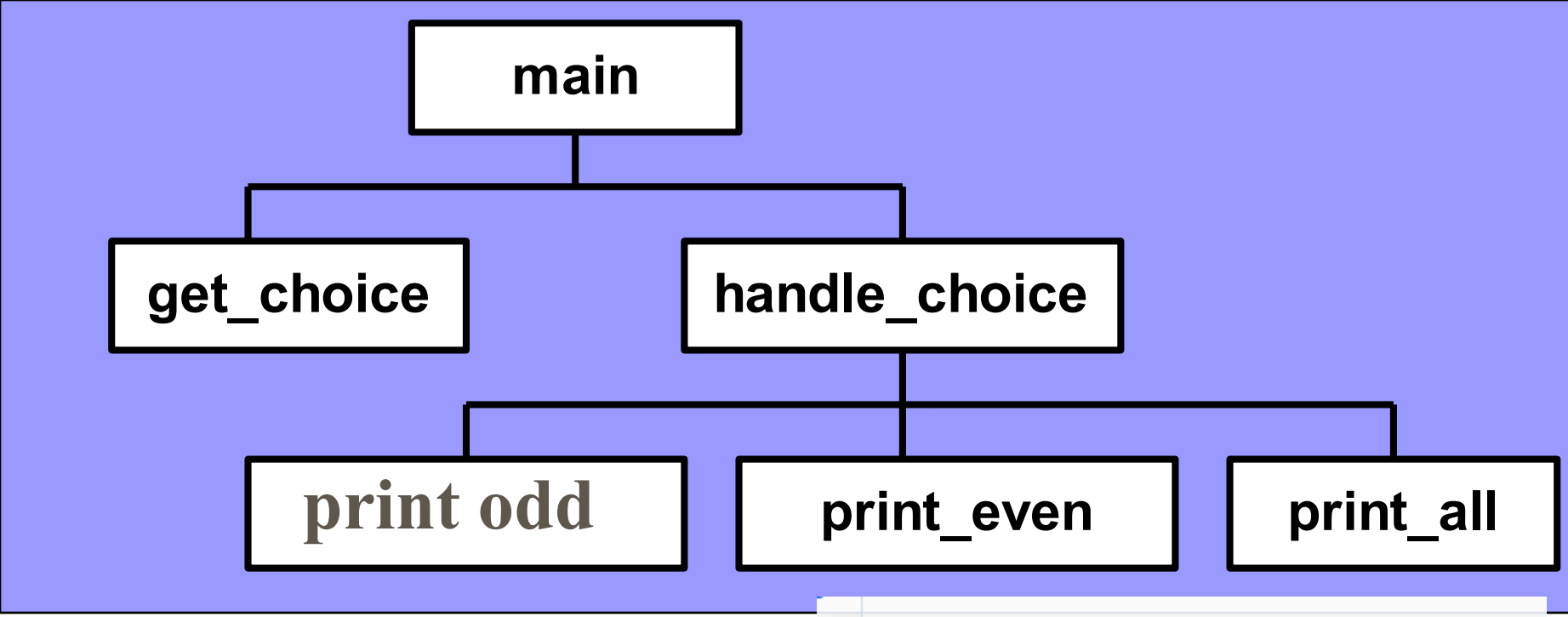


Design Issue for Functions

- Parameter Passing Method (Sec. 9.5)
  - Pass by value, by reference , by name , by Value-Result , by result
- Parameters That Are Subprograms (Sec. 9.6)
- Parameter Passing Binding (parameter order) (Sec. 9.2 – 9.4)
- Return values (Categories of Subprograms) (Sec. 9.2 – 9.4)
- Overloaded, Generic Subprograms(Sec. 9.7,9.8)
- Nested subroutine
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Nested subroutine

- **Modularity:** Subroutines help to break complex programs into smaller, more manageable parts, making them easier to understand, maintain, and modify.
  Encapsulation: Subroutines provide a way to encapsulate functionality, hiding the implementation details from other parts of the program.




```
Which series do you wish to display?

1 - Odd numbers from 1 to 30

2 - Even numbers from 1 to 30

3 - All numbers from 1 to 30

Input a number: 1
1
3
5
7
9
11
```

```cpp
#include <iostream>
using namespace std;

int getchoice() {
 int choice;  // variable for user input
 do // loop until a valid choice is entered
  {
    cout << "Which series do you wish to display?\n";
    cout << "1 - Odd numbers from 1 to 30\n";
    cout << "2 - Even numbers from 1 to 30\n";
    cout << "3 - All numbers from 1 to 30\n";
    cin >> choice;  // get choice from user
    if ((choice < 1) || (choice > 3))
     {  // if invalid entry, give message
        cout << "Choice must be 1, 2, or 3\n";
     }
  } while ((choice < 1) || (choice > 3));
  return choice;
}

void printOdd() {
   for (int i = 1; i <= 30; i = i + 2)
        cout << i << ' ';
   cout << endl;
}

void printEvent() {
   for (int i = 2; i <= 30; i = i + 2)
        cout << i << ' ';
   cout << endl;
}
```

```cpp
void printAll() {
   for (int i = 2; i <= 30; i = i + 2)
        cout << i << ' ';
   cout << endl;
}

void handlechoce(int choice)
{
    switch (choice)
    {
     case 1:
        printOdd();
       break;
     case 2:
        printEvent() ;
       break;
     case 3:
        printAll( );
       break;
    }
}

int main()
{
    int choice = getchoice();
    handlechoce(choice);
    return 0;
}
```

```
def getchoice():
    while True:
        print("Which series do you wish to display?\n");
        print("1 - Odd numbers from 1 to 30\n");
        print("2 - Even numbers from 1 to 30\n");
        print("3 - All numbers from 1 to 30\n");
        choice = input("Input a number: ")
        if choice.isdigit():
            num = int(choice);
            if ( num >= 1) and (num <= 3) :
                return num
        print("Choice must be 1, 2, or 3\n");


def handlechoice(choice):
    def printOdd() :
        for x in range(1, 30,2):
          print(x)

    def  printEvent() :
        for x in range(2, 30,2):
            print(x)

    def  printAll() :
        for x in range(1, 30,1):
            print(x)

    match choice:
        case 1:
            printOdd();
        case 2:
            printEvent() ;
        case 3:
            printAll( );

ch = getchoice()
handlechoice(ch)
```

Problem:Nested subroutine

- **Stack overflow:** If too many subroutine calls are nested or if the local variables are too large, the stack memory can overflow, causing the program to crash.

Referencing Environment

- For languages that allow nested subprograms, what referencing environment for executing the passed subprogram should be used?
  - *Shallow binding*: The environment of the call statement that enacts the passed subprogram
    - Most natural for dynamic-scoped languages
  - *Deep binding*: The environment of the definition of the passed subprogram
    - Most natural for static-scoped languages
  - *Ad hoc binding*: The environment of call statement that passed the subprogram as an actual parameter

Referencing Environment

```
function sub1() {
   var x;
   function sub2() {
       alert(x); };
   function sub3() {
       var x; x = 3;
       sub4(sub2); }
   function sub4(subx) {
       var x; x = 4;
       subx(); }
   x = 1;
   sub3();
}
```

Referencing env. of sub2():

- Shallow binding: sub4
  output = 4
- Deep binding: sub1
  output = 1
- Ad hoc binding: sub3
  output = 3

https://www.programiz.com/javascript/online-compiler

main.js

```
1  function sub1() {
2      var x=10;
3      function sub2() {   alert(x); };
4      function sub3() {
5          var x; x = 3;
6          sub4(sub2);
7      }
8      function sub4(subx) {
9          var x; x = 4;
10         subx();
11     }
12     x = 1;
13     sub3();
14 }
15 sub1();
16
```

Output

```
node /tmp/WSTHa7kNFD.js
1
```

Deep binding: The environment of the definition of the passed subprogram

Most natural for static-scoped languages

- Deep binding: sub1
  output = 1

```
2  //https://www.programiz.com/javascript/online-compiler/
3  function sub1() {
4      var x=10,y=20;
5      function sub2() {
6          //alert(x); // Creates a dialog box with the value of x
7          console.log("   sub2 ->",x);
8          console.log("   sub2 ->",y);
9      };
10     function sub3() {
11         console.log("before sub3 y ->",y);
12         console.log("before sub3 x ->",x);
13         var x;
14         var y=40;
15         x = 3;
16         sub4(sub2);
17         console.log("end-sub3 ->",x);
18     };
19     function sub4(subx) {
20         console.log("before sub4 y ->",y);
21         console.log("before sub4 x ->",x);
22         var x;
23         x = 4;
24         subx();
25         console.log("sub4 ->",x);
26     };
27    // x = 1;
28     console.log("begin-call-sub3 ->",x);
29     sub3();
30     console.log("end-sub3 ->",x);
31 };
32
33 sub1();
```

```
node /tmp/iv3YwQcChg.js
begin-call-sub3 -> 10
before sub3 y -> undefined
before sub3 x -> undefined
before sub4 y -> 20
before sub4 x -> undefined
   sub2 -> 10
   sub2 -> 20
sub4 -> 4
end-sub3 -> 3
end-sub3 -> 10
```

Overview

- Fundamentals of Subprograms (Sec. 9.2 – 9.4)
- Parameter-Passing Methods (Sec. 9.5)
- Parameters That Are Subprograms (Sec. 9.6)
- Overloaded, Generic Subprograms (Sec. 9.7,9.8)
- Design Issues for Functions (Sec. 9.9)
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

Type Checking Parameters

- Very important for reliability
- FORTRAN 77 and original C: none
- Pascal, FORTRAN 90, Java, Ada: required
- ANSI C and C++: choice is made by the user
  - Prototypes
- In Python and Ruby, variables do not have types (objects do), so parameter type checking is not possible

Design Considerations

- Two important considerations
  - Efficiency
  - One-way or two-way data transfer
- But the above considerations are in conflict
  - Good programming suggests limited access to variables, which means one-way whenever possible
  - But pass-by-reference is more efficient to pass structures of significant size

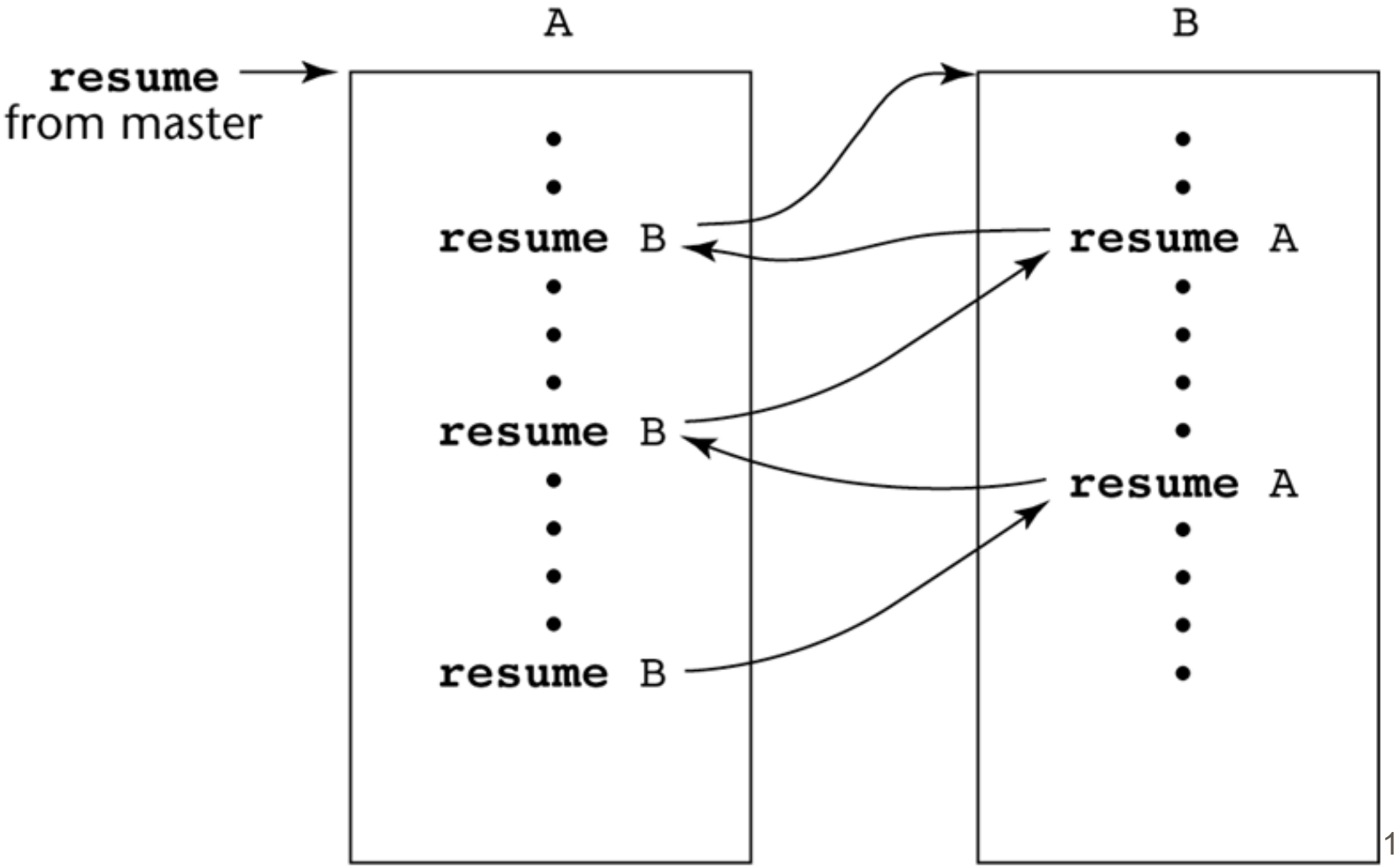
Overview

- Fundamentals of Subprograms (Sec. 9.2 – 9.4)
- Parameter-Passing Methods (Sec. 9.5)
- Parameters That Are Subprograms (Sec. 9.6)
- Overloaded, Generic Subprograms (Sec. 9.7,9.8)
- Design Issues for Functions (Sec. 9.9)
- User-Defined Overloaded Operators (Sec. 9.10)
- Coroutines (Sec. 9.11)

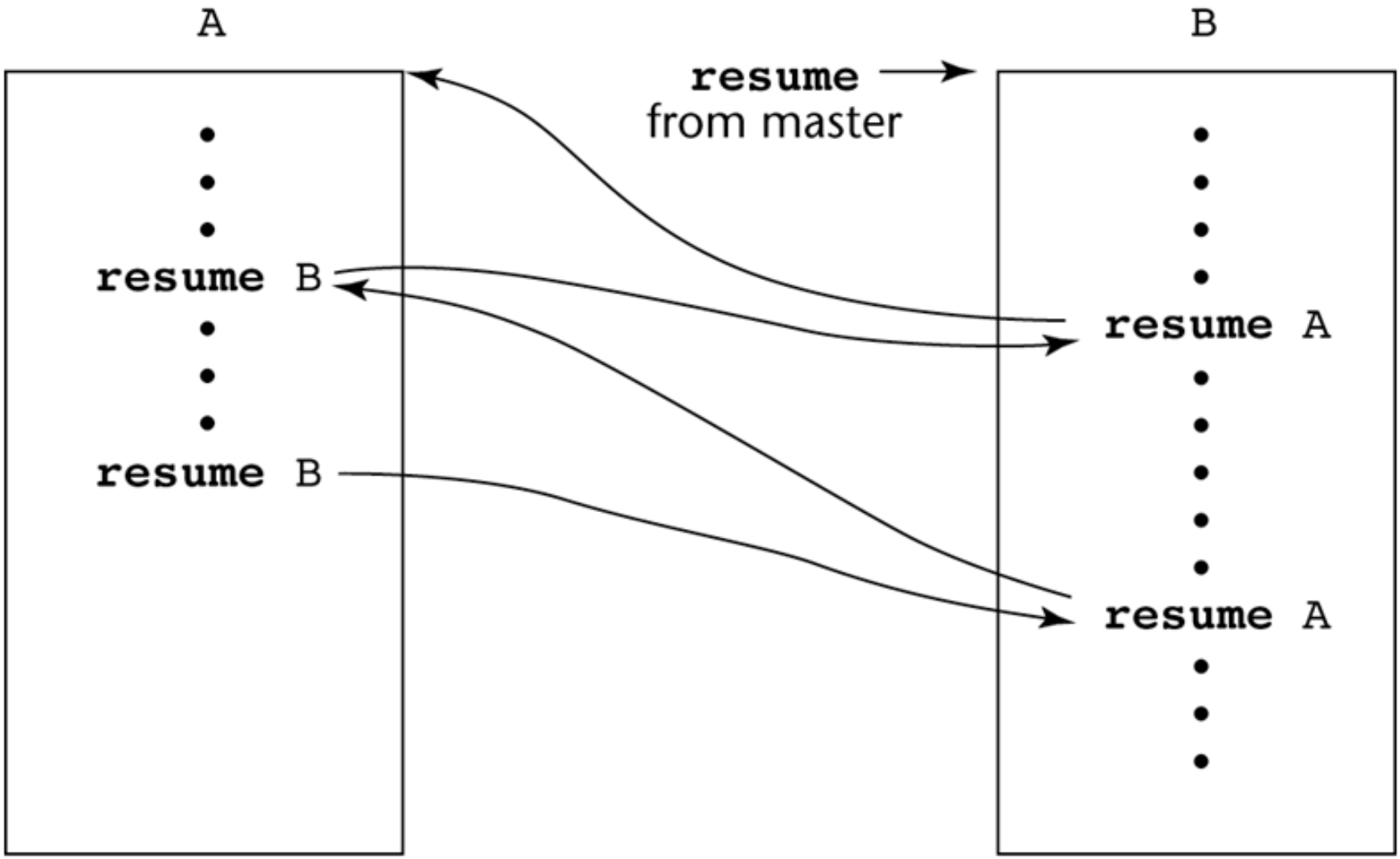
Coroutines

- A subprogram that has multiple entries and controls them itself – supported directly in Lua
  - Caller and called are on a more equal basis
- A coroutine call is named a *resume*
  - The first resume of a coroutine is to its beginning, but subsequent calls enter at the point just after the last executed statement in the coroutine
  - Repeatedly resume each other, possibly forever
- Provide *quasi-concurrent* execution of program units; execution interleaved, but not overlapped

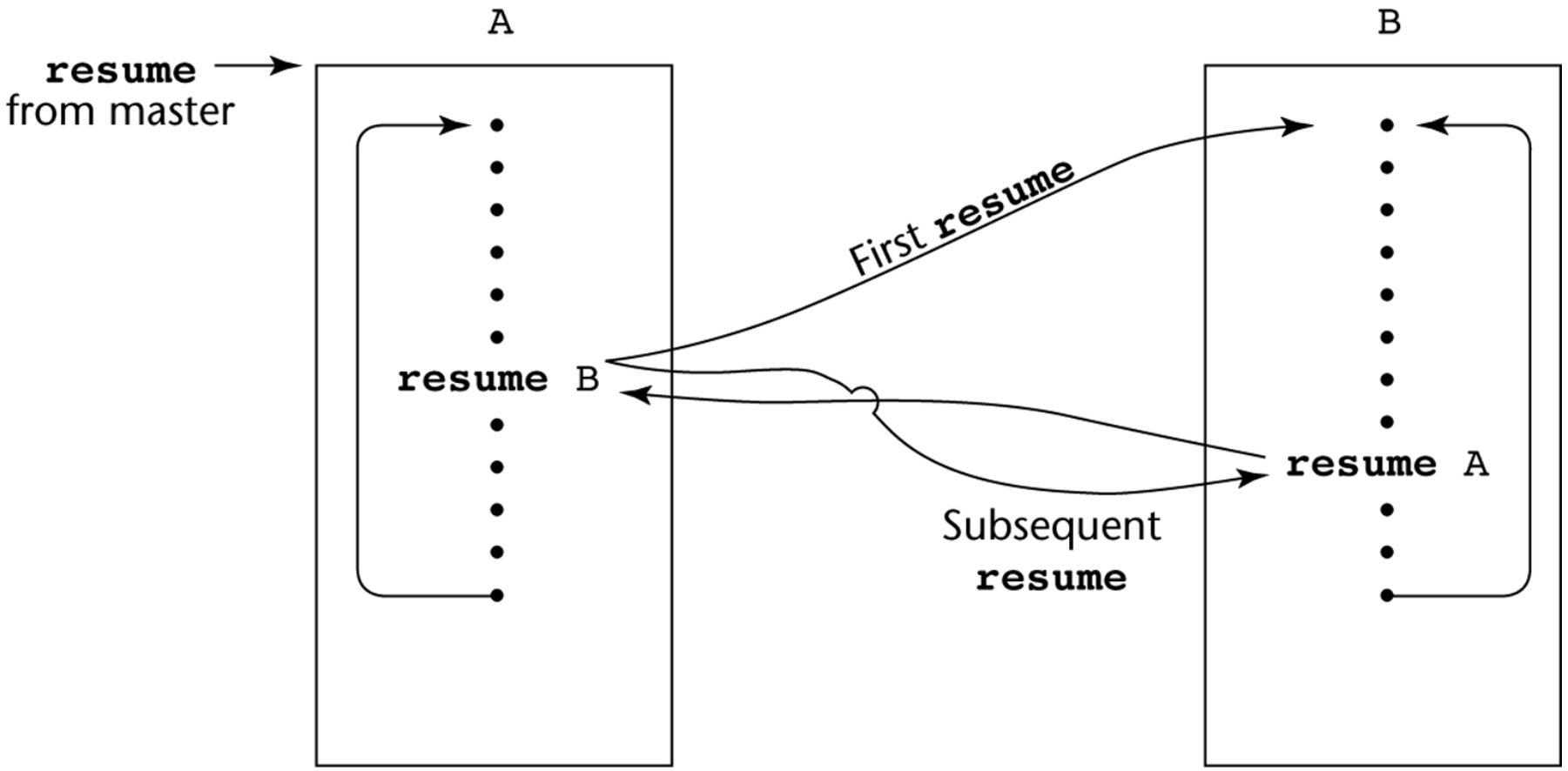
Possible Execution Controls



Possible Execution Controls



Possible Execution Controls with Loops



https://www.onlinegdb.com/online_go_compiler

```go
package main

import (
    "fmt"
    "time"
)

func f(from string) {
    //i  < 100
    for i := 0; i < 1000; i++ {
        fmt.Println(from, ":", i)
    }
}

func main() {

//    f("direct")

    go f("goroutine-1")
    go f("goroutine-2")

    go func(msg string) {
            for i := 0; i < 1000; i++ {
                fmt.Println(msg, ":", i)
            }
    }("going-3")

    time.Sleep(time.Second)
    fmt.Println("done")
}
```

Summary

- A subprogram definition describes the actions represented by the subprogram
- Subprograms can be functions or procedures
- Local variables in subprograms can be stack-dynamic or static
- Three models of parameter passing: in mode, out mode, and inout mode
- A coroutine is a special subprogram with multiple entries

Array element issues

- The implementor must choose the time to bind this parameter to an address—at the time of the call or at the time of the return.
  - If the address is computed on entry to the method, the value 12 will be returned to list[3];
  - if computed just before return, 12 will be returned to list[4].

This makes programs unportable between an implementation that chooses to evaluate the addresses for out-mode parameters at the beginning of a subprogram and one that chooses to do that evaluation at the end

```
before -> list[3] = 3 , index = 3 , list[4]=4
x = 12 , DoItIdx = 4
after -> list[3] = 12 , index = 4 , list[4]=4
```

```
1  using System;
2  class HelloWorld {
3
4    static void DoIt(  out int x , ref int DoItIdx) {
5        DoItIdx = 4;
6        x = 12;
7        Console.WriteLine("x = {0} , DoItIdx = {1}",x ,DoItIdx);
8    }
9
10
11   static void Main() {
12     int mainIdx;
13     int [] list = {0,1, 2, 3, 4, 5,6, 7, 8, 9, 10};
14     mainIdx = 3;
15     Console.WriteLine("before -> list[3] = {0} , index = {1} , list[4]={2}",list[3] ,mainIdx,list[4]);
16     DoIt(out list[ mainIdx ],ref mainIdx);
17     Console.WriteLine("after -> list[3] = {0} , index = {1} , list[4]={2}",list[3] ,mainIdx,list[4]);
18
19   }
20 }
```

```
1  -- https://www.tutorialspoint.com/compile_ada_online.php
2  with Ada.Text_IO; use Ada.Text_IO;
3  with Text_IO;
4  with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
5  procedure Hello is
6
7     list : array(0..10) of INTEGER := (0,1, 2, 3, 4, 5,6, 7, 8, 9, 10);
8     mainIdx  : Integer := 3;
9
10 PROCEDURE DoIt (x: out INTEGER ; DoItIdx : in out INTEGER  ) is
11 BEGIN
12     Put_Line("  before - > x = " & Integer'Image (x) );
13     Put_Line("  before - > DoItIdx = " & Integer'Image (DoItIdx) );
14     DoItIdx := 4;
15     Put_Line("  DoItIdx - > x = " & Integer'Image (x) );
16     x := 12;
17     Put_Line("  DoItIdx - > DoItIdx = " & Integer'Image (DoItIdx) );
18 END DoIt;
19
20 begin
21   Put_Line("before -> list[3] =  " & Integer'Image ( list(3) ) );
22   Put_Line("before -> index = " & Integer'Image (mainIdx) );
23   Put_Line("before -> list[4] =  " & Integer'Image ( list(4) ) );
24   DoIt( list ( mainIdx ) , mainIdx);
25   Put_Line("after -> list[3] =  " & Integer'Image ( list(3) ) );
26   Put_Line("after -> index = " & Integer'Image (mainIdx) );
27   Put_Line("after -> list[4] =  " & Integer'Image ( list(4) ) );
28
29 end Hello;
```

```
before -> list[3] =   3
before -> index =  3
before -> list[4] =   4
  before - > x =  32545
  before - > DoItIdx =  3
  DoItIdx - > x =  32545
  DoItIdx - > DoItIdx =  4
after -> list[3] =   12
after -> index =  4
after -> list[4] =   4
```